

Cloud Computing - Setup e Dev

Ronierison Maciel

Senac

Fevereiro de 2026



Nome: Roni

Formação: Mestre em Ciência da Computação

Ocupação: Pesquisador, Professor e Escovador de bits

Hobbies: Jogar cartas, ficar com a família

Interesses: Carros, DevSec, DS e ML

Email: ronierison.maciел@pe.senac.br

GitHub: <https://github.com/0x03c1>

- 1 Módulo 1 — Fundamentos de Computação em Nuvem
- 2 Módulo 2 — Serviços API e Arquitetura de Integração
- 3 Módulo 3 — Arquitetura de Rede em Nuvem
- 4 Módulo 4 — Armazenamento e Bancos de Dados
- 5 Módulo 5 — Setup, Deploy e Projeto Final
- 6 Conteúdo Extra
- 7 Encerramento

Objetivo da Aula

Compreender os conceitos fundamentais de computação em nuvem, sua evolução histórica e os princípios que sustentam o modelo.

Metodologia Ativa

Sala de Aula Invertida — Leitura prévia sobre a história da computação em nuvem + discussão guiada em sala.

- Definição do NIST (National Institute of Standards and Technology)
- Cinco características essenciais:
 - 1 Autoatendimento sob demanda
 - 2 Amplo acesso à rede
 - 3 Pool de recursos compartilhados
 - 4 Elasticidade rápida
 - 5 Serviço mensurado
- Diferença entre nuvem e infraestrutura tradicional on-premises

- Mainframes e time-sharing (anos 1960-70)
- Virtualização e consolidação de servidores (anos 2000)
- Surgimento da AWS (2006), Azure (2010), GCP (2011)
- Estado atual do mercado de cloud computing

Reflexão

Por que empresas migraram de data centers próprios para a nuvem? Quais foram os motivadores econômicos e técnicos?

Modelos

- **Nuvem Pública** — recursos compartilhados (Azure, AWS, GCP)
- **Nuvem Privada** — infraestrutura exclusiva de uma organização
- **Nuvem Híbrida** — combinação de pública e privada
- **Multicloud** — uso de múltiplos provedores

Azure — Exemplo

O Microsoft Azure oferece modelos híbridos nativos com **Azure Arc** e **Azure Stack**, permitindo gerenciar recursos on-premises e na nuvem de forma unificada.

Dinâmica: Think-Pair-Share

Passo 1 (5 min): Individualmente, liste 3 vantagens e 3 desvantagens de migrar um sistema legado para a nuvem.

Passo 2 (10 min): Em duplas, compare suas listas e construa uma lista unificada e priorizada.

Passo 3 (15 min): Cada dupla apresenta suas conclusões. O professor media a discussão.

Entrega

Mapa mental individual no Miro/Canva: "Ecossistema Cloud Computing".

Objetivo da Aula

Distinguir as três camadas de serviço em nuvem (IaaS, PaaS, SaaS), identificar responsabilidades de cada modelo e relacionar com serviços reais do Azure.

Metodologia Ativa

Aprendizagem Baseada em Problemas (PBL) — Cenário de uma startup que precisa escolher a camada adequada.

- O provedor fornece: servidores, rede, armazenamento, virtualização
- O cliente gerencia: SO, middleware, runtime, aplicação, dados
- Maior controle, maior responsabilidade
- Ideal para: lift-and-shift, ambientes de desenvolvimento

Serviços Azure — IaaS

- Azure Virtual Machines
- Azure Virtual Network
- Azure Disk Storage
- Azure Load Balancer

- O provedor gerencia: infraestrutura + SO + middleware + runtime
- O cliente foca em: aplicação e dados
- Produtividade elevada para desenvolvedores
- Ideal para: aplicações web, APIs, microserviços

Serviços Azure — PaaS

- Azure App Service
- Azure SQL Database
- Azure Functions
- Azure Kubernetes Service (AKS)

- O provedor gerencia tudo: infra até aplicação
- O cliente apenas utiliza o software
- Modelo de assinatura, acesso via navegador
- Ideal para: produtividade, CRM, e-mail, colaboração

Serviços Azure — SaaS

- Microsoft 365
- Microsoft Dynamics 365
- Microsoft Power BI
- Azure DevOps (parcialmente)

Componente	IaaS	PaaS	SaaS
Aplicação	Cliente	Cliente	Provedor
Dados	Cliente	Cliente	Compartilhado
Runtime	Cliente	Provedor	Provedor
Middleware	Cliente	Provedor	Provedor
Sistema Operacional	Cliente	Provedor	Provedor
Virtualização	Provedor	Provedor	Provedor
Servidores	Provedor	Provedor	Provedor
Armazenamento	Provedor	Provedor	Provedor
Rede	Provedor	Provedor	Provedor

PBL — Cenário

A startup **"TechFood"** precisa lançar um aplicativo de delivery em 3 meses. A equipe tem 2 desenvolvedores back-end e 1 front-end. O orçamento é limitado a R\$ 5.000/mês para infraestrutura.

Missão: Em grupos de 4, definam:

- 1 Qual camada de serviço é mais adequada? Justifique.
- 2 Quais serviços Azure específicos vocês utilizariam?
- 3 Monte um diagrama de arquitetura simplificado.

Entrega

Apresentação de 5 minutos por grupo + diagrama no draw.io.

Objetivo da Aula

Compreender os princípios econômicos da nuvem: CapEx vs. OpEx, economia em escala, modelos de precificação e otimização de custos.

Metodologia Ativa

Gamificação — Simulação de orçamento com a Calculadora de Preços do Azure.

CapEx (Capital Expenditure)

- Investimento inicial em hardware
- Depreciação ao longo dos anos
- Planejamento de capacidade antecipado
- Custos fixos elevados

OpEx (Operational Expenditure)

- Pagamento pelo uso (pay-as-you-go)
- Sem investimento inicial
- Escalabilidade sob demanda
- Custos variáveis e previsíveis

A nuvem transforma CapEx em OpEx.

- Provedores compram hardware em volumes massivos — custo unitário menor
- Compartilhamento de recursos entre milhares de clientes
- Eficiência energética de data centers hiperescala
- Automação reduz custos operacionais
- O benefício é repassado ao cliente via preços competitivos

Dado Real

O Azure opera mais de 60 regiões globais com milhões de servidores, permitindo que uma VM básica (B1s) custe a partir de poucos dólares/mês.

- **Pay-as-you-go** — Pague pelo que usar, sem compromisso
- **Reserved Instances** — Compromisso de 1 ou 3 anos com desconto de até 72%
- **Spot VMs** — VMs com desconto de até 90%, mas que podem ser interrompidas
- **Azure Hybrid Benefit** — Reutilizar licenças Windows Server/SQL existentes

Ferramentas de Gestão de Custos

- Azure Pricing Calculator
- Azure Cost Management + Billing
- Azure Advisor (recomendações de otimização)

Gamificação: Desafio de Orçamento

Cada grupo recebe um cenário empresarial com requisitos de infraestrutura. Usando a **Calculadora de Preços do Azure** (<https://azure.microsoft.com/pricing/calculator/>):

- 1 Estime o custo mensal da infraestrutura.
- 2 Aplique pelo menos 2 estratégias de otimização.
- 3 Compare o custo final com a estimativa de um data center on-premises equivalente.

Competição

O grupo que apresentar o menor custo com todos os requisitos atendidos vence o desafio!

Objetivo da Aula

Analisar estratégias de migração, desafios e barreiras para adoção de arquiteturas em nuvem.

Metodologia Ativa

Estudo de Caso — Análise de migrações reais e seus desafios (Netflix, Nubank, Magazine Luiza).

- ❶ **Rehost** (Lift-and-Shift) — Mover sem alterações
- ❷ **Replatform** (Lift-and-Reshape) — Pequenas otimizações
- ❸ **Refactor** — Re-arquitetar para cloud-native
- ❹ **Repurchase** — Substituir por SaaS
- ❺ **Retire** — Descomissionar sistemas obsoletos
- ❻ **Retain** — Manter on-premises (por enquanto)

Azure Migrate

Ferramenta do Azure que avalia, planeja e executa migrações de servidores, bancos de dados, aplicações e dados.

Desafios Técnicos

- Dependência de tecnologias legadas
- Incompatibilidade de arquiteturas
- Latência e conectividade
- Segurança e conformidade
- Vendor lock-in

Barreiras Organizacionais

- Resistência cultural à mudança
- Falta de profissionais qualificados
- Custos de migração subestimados
- Regulamentações (LGPD, setoriais)
- Falta de governança cloud

Estudo de Caso: Migração de Sistema Legado

Uma empresa de varejo possui um ERP monolítico rodando em servidores físicos Windows Server 2012 com SQL Server 2014. O sistema sofre com: indisponibilidade em picos de Black Friday, alto custo de manutenção e dificuldade de integração com e-commerce.

Missão (em grupos):

- 1 Qual estratégia de migração (6 R's) vocês recomendam? Por quê?
- 2 Quais serviços Azure seriam utilizados?
- 3 Identifiquem 3 riscos e proponham mitigações.
- 4 Elaborem um cronograma simplificado de migração.

Entrega

Documento de Plano de Migração (1 página) + apresentação de 7 min.

Objetivo da Aula

Compreender o conceito de API (Application Programming Interface), os paradigmas REST, GraphQL e gRPC, e a importância das APIs na economia digital e na arquitetura cloud.

Metodologia Ativa

Hands-on Lab — Consumo e teste de APIs públicas usando Postman/Thunder Client, seguido de exploração no Portal Azure.

Duração

2 horas-aula

O que são APIs e por que importam?

- **API** = Interface de Programação de Aplicações
- Contrato formal que define como dois sistemas se comunicam
- Abstração: o consumidor não precisa conhecer a implementação interna
- Habilitam:
 - Integração entre microsserviços
 - Automação de processos
 - Exposição de funcionalidades para parceiros/clientes
 - Monetização de dados e serviços (*API Economy*)

Contexto Cloud

Na nuvem, **tudo é API**. O próprio Azure Portal é um frontend que consome as APIs REST do Azure Resource Manager (ARM).

- APIs como produto: empresas monetizam acesso a dados e funcionalidades
- Exemplos reais:
 - **Stripe** — API de pagamentos (avaliada em bilhões)
 - **Twilio** — API de comunicação (SMS, voz, vídeo)
 - **Google Maps API** — Geolocalização como serviço
 - **Open Banking Brasil** — APIs reguladas pelo Banco Central
- Marketplaces de APIs: RapidAPI, Azure Marketplace, AWS Marketplace

Reflexão

O Open Banking obriga bancos brasileiros a expor APIs padronizadas. Qual o impacto disso para fintechs e consumidores?

- Baseado no protocolo HTTP
- Utiliza verbos semânticos:
 - GET — Ler recurso
 - POST — Criar recurso
 - PUT — Atualizar recurso (completo)
 - PATCH — Atualizar parcialmente
 - DELETE — Remover recurso
- Recursos identificados por URIs
- Stateless: cada requisição é independente
- Formato padrão: JSON

Exemplo de Endpoints

```
GET /api/v1/products
GET /api/v1/products/42
POST /api/v1/products
PUT /api/v1/products/42
DELETE /api/v1/products/42
```

Códigos HTTP Comuns

```
200 OK | 201 Created
400 Bad Request
401 Unauthorized
404 Not Found
500 Internal Server Error
```

GraphQL

- Desenvolvido pelo Facebook (2015)
- Cliente define exatamente os dados que precisa
- Endpoint único (/graphql)
- Elimina *over-fetching* e *under-fetching*
- Schema tipado e autodocumentado
- Ideal para: frontends complexos, mobile

gRPC

- Desenvolvido pelo Google
- Protocol Buffers (formato binário)
- HTTP/2 nativo (multiplexação)
- Alta performance e baixa latência
- Streaming bidirecional
- Ideal para: comunicação entre microsserviços

Aspecto	REST	GraphQL	gRPC
Formato	JSON	JSON	Protobuf (binário)
Protocolo	HTTP/1.1+	HTTP/1.1+	HTTP/2
Curva de aprendizado	Baixa	Média	Alta

- **Consistência** — Padrões de nomenclatura e comportamento uniformes
- **Versionamento** — /api/v1/, /api/v2/ para evitar quebras
- **Documentação** — OpenAPI/Swagger como padrão de mercado
- **Paginação** — Limitar retorno de grandes conjuntos de dados
- **Tratamento de erros** — Mensagens claras com códigos HTTP adequados
- **Segurança** — Autenticação (OAuth 2.0, API Keys), HTTPS obrigatório
- **Rate Limiting** — Proteger contra abuso (ex: 100 req/min)
- **HATEOAS** — Links para navegação entre recursos (maturidade REST)

Modelo de Maturidade de Richardson

Nível 0 (RPC) → Nível 1 (Recursos) → Nível 2 (Verbos HTTP) → Nível 3 (HATEOAS). A maioria das APIs em produção opera no Nível 2.

Hands-on: Explorando APIs com Postman

- ❶ Instale o **Postman** (desktop) ou **Thunder Client** (VS Code).
- ❷ Consuma a API pública <https://jsonplaceholder.typicode.com/>:
 - GET /posts — Liste todos os posts
 - GET /posts/1 — Obtenha um post específico
 - POST /posts — Crie um novo post (com body JSON)
 - PUT /posts/1 — Atualize um post
 - DELETE /posts/1 — Delete um post
- ❸ Para cada operação, registre: método, URL, body (se houver), status code e resposta.

Desafio Extra

Acesse <https://editor.swagger.io/> e crie uma especificação OpenAPI para uma API fictícia de gerenciamento de tarefas (mínimo 4 endpoints).

Objetivo da Aula

Aprender a provisionar e gerenciar serviços de nuvem utilizando o Portal Azure (interface gráfica), compreendendo o Azure Resource Manager como camada unificada de gerenciamento.

Metodologia Ativa

Laboratório Guiado (GUI) — Provisionamento passo a passo de recursos via Portal Azure.

- Camada de gerenciamento unificada de **todos** os recursos Azure
- Toda ação no Azure (Portal, CLI, SDK, API) passa pelo ARM
- Funcionalidades do ARM:
 - Criação, atualização e exclusão de recursos
 - Controle de acesso (RBAC)
 - Tags para organização
 - Locks para proteção contra exclusão acidental
 - Templates declarativos (ARM Templates, Bicep)

Analogia

O ARM é como um "receptionista universal": não importa por qual porta você entre (Portal, CLI, API), ele sempre processa seu pedido da mesma forma.

Acesso: <https://portal.azure.com>

Elementos principais da interface:

- **Barra superior** — Pesquisa global, Cloud Shell, notificações
- **Menu lateral** — Favoritos, todos os serviços
- **Dashboard** — Painel personalizável
- **Resource Groups** — Agrupamentos lógicos de recursos
- **Breadcrumb** — Navegação hierárquica dentro dos recursos

Portal Azure → Pesquisar "Resource groups" → + **Create**

- 1 **Subscription:** Selecione sua assinatura (Free/Student)
- 2 **Resource group:** Digite rg-curso-cloud
- 3 **Region:** Selecione Brazil South
- 4 Clique em **Review + Create** → **Create**

Boas Práticas de Nomenclatura

Padrão recomendado: rg-<projeto>-<ambiente>-<região>

Exemplos: rg-ecommerce-prod-brazilsouth, rg-api-dev-eastus

Portal → Pesquisar "App Services" → + **Create** → **Web App**

- 1 **Resource Group:** rg-curso-cloud
- 2 **Name:** app-curso-<seunome>-2026 (deve ser único globalmente)
- 3 **Publish:** Code
- 4 **Runtime stack:** Node 18 LTS (ou Python 3.11)
- 5 **Region:** Brazil South
- 6 **Pricing plan:** Free F1
- 7 **Review** + **Create** → **Create**

Importante

O tier **Free F1** é gratuito e suficiente para as práticas do curso.

Comandos via Azure CLI

```
az group create -name rg-curso-cloud -location brazilsouth  
  
az appservice plan create -name plan-curso  
-resource-group rg-curso-cloud -sku F1 -is-linux  
  
az webapp create -name app-curso-cloud-2026  
-resource-group rg-curso-cloud -plan plan-curso  
-runtime "NODE:18-lts"
```

Os comandos CLI são úteis para automação e scripts, mas nas práticas deste curso utilizaremos exclusivamente o **Portal Azure (GUI)**.

Lab (Portal Azure): Provisionamento de Recursos

Individualmente, pelo Portal Azure:

- 1 Crie um Resource Group chamado `rg-aula06-<seunome>`.
- 2 Adicione **Tags**: `Disciplina=CloudComputing`, `Aluno=<nome>`.
- 3 Provisione um **App Service** (Web App) com runtime Node 18 LTS, tier Free F1.
- 4 Acesse a URL padrão da aplicação e verifique a página inicial.
- 5 No painel do App Service, explore:
 - **Overview** — métricas e URL
 - **Configuration** — variáveis de ambiente
 - **Deployment Center** — opções de deploy
- 6 Ao final, **exclua** o Resource Group para evitar custos.

Entrega

Documento com print screen de cada etapa + reflexão de 5 linhas sobre a experiência.

Objetivo da Aula

Compreender o papel do API Gateway na arquitetura de microserviços, explorar o Azure API Management e plataformas de integração.

Metodologia Ativa

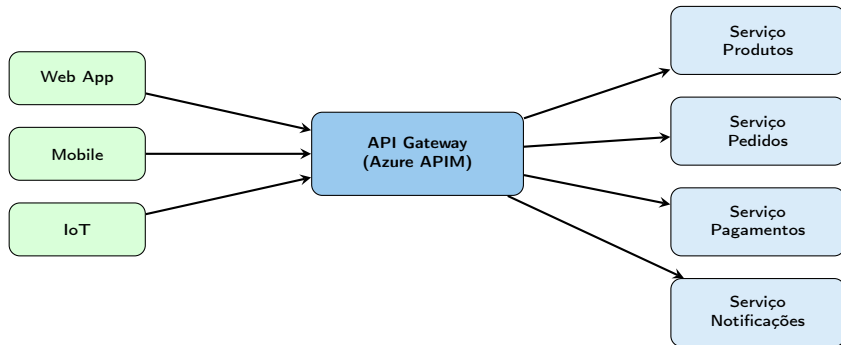
Aprendizagem por Descoberta — Exploração guiada do Azure API Management no Portal.

Sem Gateway

- Clientes conhecem cada microserviço
- Autenticação duplicada em cada serviço
- Sem controle centralizado de tráfego
- Dificuldade de monitoramento
- Acoplamento direto cliente-serviço
- Exposição desnecessária de endpoints

Com Gateway

- Ponto de entrada único
- Autenticação centralizada
- Rate limiting e throttling
- Monitoramento unificado
- Desacoplamento total
- Transformação de requisições



- Serviço gerenciado para publicar, proteger e monitorar APIs
- Componentes:
 - **API Gateway** — Proxy que recebe e roteia chamadas
 - **Developer Portal** — Portal de documentação para consumidores
 - **Management Plane** — Configuração e administração
- **Políticas (Policies):** Regras aplicadas a cada requisição
 - Rate limiting e quotas
 - Validação de JWT tokens
 - Transformação de headers e body
 - Caching de respostas
 - CORS (Cross-Origin Resource Sharing)

Portal → Pesquisar "API Management" → + **Create**

Tier recomendado para estudo: **Consumption** (paga apenas por uso).

- **Azure Logic Apps** — Workflows visuais de integração (low-code)
 - Centenas de conectores prontos (Office 365, Salesforce, SAP, etc.)
 - Triggers automáticos (e-mail recebido, arquivo criado, timer)
- **Azure Service Bus** — Mensageria empresarial (filas e tópicos)
- **Azure Event Grid** — Roteamento de eventos reativo
- **Azure Functions** — Computação serverless orientada a eventos

Padrões de Integração

- **Request/Response** — Síncrono (REST, gRPC)
- **Publish/Subscribe** — Assíncrono (Service Bus Topics, Event Grid)
- **Event-Driven** — Reativo (Functions + Event Grid)
- **Saga Pattern** — Transações distribuídas entre microsserviços

Lab (Portal Azure): Explorando Azure API Management

- 1 No Portal, crie uma instância de **API Management** (tier Consumption).
- 2 Em "APIs" → + **Add API** → selecione "Demo Conference API" (API de exemplo da Microsoft).
- 3 Teste a API diretamente pelo Portal: aba **Test** → selecione uma operação GET → clique em **Send**.
- 4 Em **Design** → selecione uma operação → **Inbound processing** → adicione política de **Rate limit** (3 chamadas por 60 segundos).
- 5 Teste novamente e observe o comportamento de throttling (status 429).
- 6 Explore o **Developer Portal** e a documentação gerada automaticamente.

Entrega

Relatório com prints dos testes + explicação das políticas configuradas.

Objetivo da Aula

Explorar modelos de escalabilidade na nuvem, serverless computing, containers e tecnologias emergentes de integração.

Metodologia Ativa

Jigsaw (Quebra-cabeça) — Cada grupo estuda uma tecnologia e ensina aos demais.

Escala Vertical (Scale-Up)

- Aumentar CPU, RAM, disco da máquina
- Limite físico do hardware
- Simples de implementar
- Pode exigir downtime para redimensionar
- Ex: Mudar de B1s para D2s_v3

Escala Horizontal (Scale-Out)

- Adicionar mais instâncias idênticas
- Teoricamente ilimitado
- Requer aplicação *stateless*
- Balanceador de carga distribui tráfego
- Autoscaling automático no Azure

App Service → **Scale up** (vertical) ou **Scale out** (horizontal)

Configure regras de autoscale baseadas em CPU, memória ou métricas custom.

- Execução de código sob demanda, sem gerenciar servidores
- Cobrança por execução (não por tempo ocioso)
- Escala automaticamente de zero a milhares de instâncias
- **Triggers** (gatilhos): HTTP, Timer, Blob, Queue, Cosmos DB, Event Grid
- **Bindings** (vinculações): entrada e saída automáticas para outros serviços
- Linguagens suportadas: C#, JavaScript, Python, Java, PowerShell, TypeScript

Casos de Uso

- API backend leve para aplicações mobile
- Processamento de imagens ao upload em Blob Storage
- Automação de tarefas agendadas (limpeza de dados, relatórios)
- Webhooks e integrações entre serviços

Containers (Docker)

- Empacotam aplicação + dependências
- Portabilidade total entre ambientes
- Mais leves que VMs (compartilham kernel)
- Imagens imutáveis e versionadas
- Docker Hub / Azure Container Registry

Kubernetes (AKS)

- Orquestração de containers em escala
- Auto-healing e auto-scaling
- Rolling updates e rollbacks
- Service discovery e load balancing
- Azure Kubernetes Service (gerenciado)

Container Apps

O **Azure Container Apps** é uma alternativa mais simples ao AKS para quem quer containers sem a complexidade do Kubernetes — ideal para microsserviços e APIs.

- **Edge Computing** — Azure IoT Edge
 - Processamento de dados na borda (próximo à fonte)
 - Redução de latência e dependência de conectividade
- **IA como Serviço** — Azure AI Services
 - APIs pré-treinadas: visão computacional, linguagem natural, fala
 - Azure OpenAI Service: GPT-4, DALL-E, Whisper como serviço
- **Low-Code / No-Code** — Power Platform
 - Power Apps, Power Automate, Power BI
 - Citizen developers criando soluções sem código

Jigsaw: Tecnologias Emergentes

Divida a turma em 4 **grupos-base**. Cada membro do grupo vai para um **grupo de especialistas** diferente:

- ❶ **Especialistas A:** Serverless (Azure Functions) — explore no Portal
- ❷ **Especialistas B:** Containers (Docker + Azure Container Apps)
- ❸ **Especialistas C:** Edge Computing (Azure IoT Hub)
- ❹ **Especialistas D:** IA como Serviço (Azure AI Services)

30 min de estudo no grupo de especialistas → Retornam ao grupo-base e ensinam sua tecnologia (10 min cada).

Entrega

Quadro comparativo das 4 tecnologias: casos de uso, vantagens, limitações e serviços Azure.

Objetivo da Aula

Compreender ameaças comuns a APIs, mecanismos de autenticação e autorização, e boas práticas de segurança no Azure.

Metodologia Ativa

Estudo de Caso + Hands-on — Análise de vulnerabilidades reais (OWASP API Top 10) e configuração de segurança no Portal.

Conteúdo de Reserva

i Esta aula é conteúdo extra/reserva para complementar o módulo de APIs caso haja tempo disponível.

- ❶ **Broken Object Level Authorization** — Acesso a dados de outros usuários
- ❷ **Broken Authentication** — Falhas na autenticação
- ❸ **Broken Object Property Level Authorization** — Exposição de propriedades sensíveis
- ❹ **Unrestricted Resource Consumption** — Ausência de rate limiting
- ❺ **Broken Function Level Authorization** — Acesso a funções restritas
- ❻ **Unrestricted Access to Sensitive Flows** — Automação maliciosa
- ❼ **Server-Side Request Forgery (SSRF)** — Requisições forjadas
- ❽ **Security Misconfiguration** — Configurações inseguras
- ❾ **Improper Inventory Management** — APIs não documentadas expostas
- ❿ **Unsafe Consumption of APIs** — Confiança cega em APIs de terceiros

Autenticação (Quem é você?)

- **API Keys** — Simples, mas limitado
- **OAuth 2.0** — Padrão da indústria
- **OpenID Connect** — Identidade sobre OAuth 2.0
- **JWT (JSON Web Tokens)** — Tokens autocontidos
- **Azure Entra ID** — Identity provider do Azure

Autorização (O que pode fazer?)

- **RBAC** — Controle baseado em funções
- **ABAC** — Controle baseado em atributos
- **Scopes** — Permissões granulares em OAuth
- **Azure RBAC** — Roles atribuídas a recursos
- **Principle of Least Privilege**

API Management → Seleccione uma API → **Settings**:

- **Subscription required:** Exige chave de assinatura para consumir
- **OAuth 2.0:** Configure servidor de autorização (Azure Entra ID)
- **Client certificates:** Autenticação mútua (mTLS)

Policies → **Inbound**:

- `validate-jwt` — Validar tokens JWT
- `ip-filter` — Permitir/bloquear IPs
- `cors` — Configurar Cross-Origin

Estudo de Caso: Incidentes de Segurança em APIs

Em grupos de 4:

- 1 O professor distribui 3 casos reais de incidentes de segurança envolvendo APIs (ex: vazamento de dados por falta de autenticação, ataque por rate limiting ausente, etc.).
- 2 Cada grupo analisa: qual vulnerabilidade (OWASP Top 10), como foi explorada, qual o impacto e como prevenir.
- 3 Apresentação: 5 minutos por grupo.

Hands-on Extra (se houver tempo)

No API Management, configure a política `validate-jwt` para exigir um token válido em uma API de teste.

Objetivo da Aula

Compreender os conceitos de rede em ambientes cloud: Virtual Networks, sub-redes, Network Security Groups e endereçamento IP no Azure.

Metodologia Ativa

Laboratório Hands-on (GUI) — Criação e configuração de redes virtuais no Portal Azure.

- **Endereçamento IP** — IPv4 (32 bits), IPv6 (128 bits)
- **CIDR Notation** — Ex: $10.0.0.0/16 = 65.536$ endereços
- **Sub-redes** — Divisão lógica de uma rede maior
- **Roteamento** — Como pacotes encontram o destino
- **DNS** — Resolução de nomes para endereços IP
- **Firewall** — Filtro de tráfego por regras

Cálculo rápido de sub-redes

$/16 = 65.536$ IPs | $/24 = 256$ IPs | $/28 = 16$ IPs

Azure reserva 5 IPs por sub-rede (primeiro, último e 3 de gerenciamento).

- Bloco fundamental de rede privada isolada no Azure
- Cada VNet tem um espaço de endereçamento CIDR (ex: 10.0.0.0/16)
- Recursos dentro de uma VNet podem se comunicar entre si
- Recursos em VNets diferentes são isolados por padrão
- Uma VNet está vinculada a uma **região** Azure

Componentes de Rede no Azure

- **Sub-redes (Subnets)** — Segmentação dentro da VNet
- **NIC (Network Interface)** — Conecta VM à sub-rede
- **NSG (Network Security Group)** — Firewall de camada 4 (TCP/UDP)
- **Route Tables (UDR)** — Tabelas de roteamento customizadas
- **Public IP** — Endereço IP público para acesso externo

Portal → Pesquisar "Virtual networks" → + **Create**

Aba Basics:

- 1 Resource Group: `rg-curso-cloud`
- 2 Name: `vnet-curso`
- 3 Region: Brazil South

Aba IP Addresses:

- 1 Address space: `10.0.0.0/16`
- 2 + Add subnet → Name: `subnet-web`, Range: `10.0.1.0/24`
- 3 + Add subnet → Name: `subnet-db`, Range: `10.0.2.0/24`

Review + **Create** → **Create**

- Firewall de camada 4 para filtrar tráfego de entrada e saída
- Regras baseadas em: prioridade, direção, origem, destino, porta, protocolo, ação
- Pode ser associado a **sub-redes** ou **NICs** individuais
- Regras padrão: negar todo tráfego de entrada, permitir tráfego interno da VNet

Portal → Pesquisar "Network security groups" → + **Create**

Após criação → **Inbound security rules** → + **Add**:

- Source: Any | Destination: Any | Port: 80 | Protocol: TCP | Action: Allow | Priority: 100
| Name: Allow-HTTP

Depois: **Subnets** → + **Associate** → vincule à subnet-web.

Criação da VNet

```
az network vnet create -name vnet-curso  
-resource-group rg-curso-cloud -location brazilsouth  
-address-prefix 10.0.0.0/16  
-subnet-name subnet-web -subnet-prefix 10.0.1.0/24
```

Criação do NSG

```
az network nsg create -name nsg-web  
-resource-group rg-curso-cloud
```

Regra de Liberação HTTP

```
az network nsg rule create -nsg-name nsg-web  
-resource-group rg-curso-cloud  
-name Allow-HTTP -priority 100  
-destination-port-ranges 80  
-protocol Tcp -access Allow
```

Lab (Portal Azure): Arquitetura de Rede

- ❶ Crie uma VNet `vnet-aula10` com espaço `10.0.0.0/16`.
- ❷ Crie 3 sub-redes: `subnet-web (/24)`, `subnet-app (/24)`, `subnet-db (/24)`.
- ❸ Crie 2 NSGs:
 - `nsg-web`: permitir HTTP (80) e HTTPS (443) de qualquer origem.
 - `nsg-db`: permitir porta 3306 apenas da sub-rede `subnet-app`.
- ❹ Associe os NSGs às sub-redes correspondentes.
- ❺ Tire print de: VNet com sub-redes, regras de cada NSG, associações.

Entrega

Diagrama da topologia (draw.io ou papel) + prints do Portal.

Objetivo da Aula

Explorar Software Defined Networking (SDN), topologias de rede em nuvem e serviços avançados de rede do Azure.

Metodologia Ativa

Mapa Conceitual Colaborativo — Construção coletiva usando Miro, draw.io ou quadro branco.

- Separação entre **plano de controle** (decisões de roteamento) e **plano de dados** (encaminhamento de pacotes)
- Rede gerenciada por software, não por configuração manual de hardware
- Toda rede em nuvem pública é SDN
- Benefícios:
 - Provisionamento programático (via Portal, API, IaC)
 - Automação de políticas de segurança
 - Visibilidade e monitoramento centralizado
 - Escalabilidade sem limites físicos

Hub-Spoke (Mais Comum)

- VNet "Hub" centraliza serviços compartilhados (Firewall, VPN, DNS)
- VNets "Spoke" contêm workloads isolados
- Comunicação via VNet Peering
- Recomendada pela Microsoft

Outros Padrões

- **Mesh** — Todas as VNets conectadas entre si
- **VNet Peering** — Conexão direta entre VNets (mesma ou diferentes regiões)
- **Service Endpoints** — Acesso seguro a PaaS pela VNet
- **Private Link** — IP privado para serviços PaaS

- **Azure Load Balancer** — Balanceamento L4 (TCP/UDP), público ou interno
- **Azure Application Gateway** — Balanceamento L7 (HTTP/HTTPS) + WAF
- **Azure Front Door** — CDN global + aceleração + WAF
- **Azure Firewall** — Firewall gerenciado, stateful, com threat intelligence
- **Azure DDoS Protection** — Proteção contra ataques volumétricos
- **Azure DNS** — Hospedagem de zonas DNS
- **Azure Traffic Manager** — Roteamento DNS entre regiões (failover, performance)
- **Azure Bastion** — Acesso RDP/SSH seguro sem IP público

Quando usar cada serviço de balanceamento?

Serviço	Camada	Global?	WAF?	Melhor para
Load Balancer	L4	Não	Não	VMs e alta performance TCP/UDP
App Gateway	L7	Não	Sim	Apps web regionais
Front Door	L7	Sim	Sim	Apps web globais + CDN
Traffic Manager	DNS	Sim	Não	Failover entre regiões

Mapa Conceitual Colaborativo

Em grupos de 5, construam um mapa conceitual que relacione:

- 1 Todos os serviços de rede do Azure estudados.
- 2 A camada OSI em que cada serviço opera.
- 3 Cenários de uso reais para cada serviço.
- 4 Como os serviços se interconectam (ex: Front Door → App Gateway → VM).

Ferramenta: Miro, draw.io ou quadro branco físico.

Entrega

Exportação do mapa em PDF/imagem + apresentação de 5 min por grupo.

Objetivo da Aula

Compreender estratégias de conectividade híbrida e multicloud, incluindo VPN Gateway, ExpressRoute e Azure Arc.

Metodologia Ativa

Debate Estruturado — "Multicloud vs. Single Cloud: qual a melhor estratégia?"

- **Azure VPN Gateway** — Túnel criptografado (IPSec/IKE)
 - **Site-to-Site (S2S):** Conecta rede on-premises à VNet
 - **Point-to-Site (P2S):** Conecta dispositivo individual à VNet
 - **VNet-to-VNet:** Conecta duas VNets via VPN
- **Azure ExpressRoute** — Conexão privada dedicada
 - Não passa pela internet pública
 - Latência previsível e alta largura de banda (até 100 Gbps)
 - Ideal para cargas sensíveis (financeiro, saúde)
- **Azure Virtual WAN** — Hub de rede gerenciado
 - Conecta branches, VNets, VPN e ExpressRoute em um hub unificado

Vantagens do Multicloud

- Evita *vendor lock-in*
- Best-of-breed por provedor
- Resiliência contra falhas regionais
- Conformidade regulatória (dados em regiões específicas)
- Poder de negociação com provedores

Desafios do Multicloud

- Complexidade operacional elevada
- Equipe precisa dominar múltiplos provedores
- Custos de transferência de dados (egress)
- Segurança e governança fragmentadas
- Integração e portabilidade de aplicações

Azure Arc

Gerenciamento unificado de recursos em Azure, on-premises, AWS e GCP. Permite aplicar políticas Azure, RBAC e monitoramento em qualquer lugar.

Debate Estruturado

A turma se divide em dois grupos:

Grupo 1 — Defende Multicloud: Prepare 5 argumentos sólidos com exemplos reais de empresas que adotaram multicloud.

Grupo 2 — Defende Single Cloud: Prepare 5 argumentos sólidos com exemplos reais de empresas que optaram por um único provedor.

Dinâmica: 15 min preparação → 5 min apresentação por grupo → 10 min réplica/tréplica → 10 min síntese do professor.

Entrega

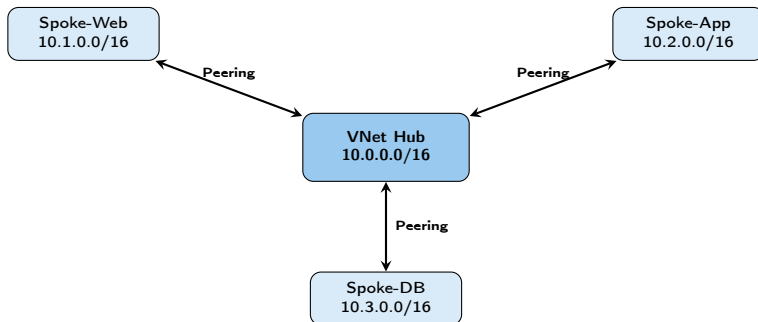
Texto individual (meia página): "Minha posição fundamentada sobre multicloud após o debate".

Objetivo da Aula

Aplicar todos os conceitos de rede em um cenário completo: configurar uma arquitetura hub-spoke funcional no Portal Azure.

Metodologia Ativa

Projeto Prático Guiado (GUI) — Construção incremental no Portal Azure.



1 Criar VNets:

Portal → Virtual networks → + Create

Crie: `vnet-hub` (10.0.0.0/16), `vnet-spoke-web` (10.1.0.0/16), `vnet-spoke-app` (10.2.0.0/16)

2 Configurar Peering:

`vnet-hub` → Peerings → + Add

Peering name: `hub-to-spoke-web`

Remote VNet: `vnet-spoke-web`

Repita para cada spoke.

3 Criar NSGs e associar às sub-redes

4 Validar: Verifique que o status do peering é "Connected"

Projeto: Arquitetura Hub-Spoke Completa

Em duplas, pelo Portal Azure:

- 1 Crie a VNet Hub e 2 VNets Spoke (conforme slide anterior).
- 2 Configure VNet Peering bidirecional (Hub↔Spoke).
- 3 Em cada Spoke, crie uma sub-rede com NSG adequado.
- 4 Desenhe o diagrama completo da topologia (draw.io).
- 5 Documente: espaços de endereçamento, regras NSG, status dos peerings.

Entrega

Relatório técnico com: diagrama, prints de cada etapa, análise de segurança e sugestões de melhoria.

Objetivo da Aula

Aprofundar em serviços de DNS, CDN (Content Delivery Network) e técnicas de otimização de performance de rede no Azure.

Metodologia Ativa

Rotação por Estações — Cada estação explora um serviço diferente no Portal Azure.

Conteúdo de Reserva

i Aula extra para aprofundamento em performance de rede, caso haja tempo no módulo.

- Hospedagem de zonas DNS no Azure (alta disponibilidade via anycast)
- **DNS Público** — Resolução de domínios externos (ex: `meusite.com.br`)
- **DNS Privado** — Resolução de nomes dentro de VNets (sem exposição pública)
- Integração nativa com outros serviços Azure

Azure CDN e Front Door

- **CDN** — Cache de conteúdo estático em pontos de presença (PoPs) globais
- **Front Door** — CDN + balanceamento L7 + WAF + aceleração dinâmica
- Redução de latência para usuários globais
- Cache inteligente com regras de expiração

Rotação por Estações (25 min cada)

Estação 1 — Azure DNS: Explore a criação de uma zona DNS privada no Portal e vincule a uma VNet.

Estação 2 — Front Door: Explore as opções de configuração do Azure Front Door (documentação + Portal).

Estação 3 — Network Watcher: Use o **Network Watcher** no Portal para diagnosticar conectividade (IP Flow Verify, NSG Diagnostics).

Cada grupo documenta: o que o serviço faz, como configurar, e para que cenário seria ideal.

Objetivo da Aula

Compreender os modelos de armazenamento em nuvem: Blob, File, Queue, Table e Disk Storage no Azure.

Metodologia Ativa

Rotação por Estações (GUI) — Cada estação explora um tipo de armazenamento no Portal Azure.

Azure Storage Account

- **Blob Storage** — Objetos não estruturados (imagens, vídeos, backups, logs)
- **File Storage** — Compartilhamento de arquivos SMB/NFS (substituir file server)
- **Queue Storage** — Filas de mensagens (comunicação assíncrona)
- **Table Storage** — NoSQL key-value simples e barato

Camadas de Acesso (Blob)

- **Hot** — Acesso frequente (custo de armazenamento mais alto, custo de acesso mais baixo)
- **Cool** — Acesso infrequente, mín. 30 dias
- **Cold** — Acesso raro, mín. 90 dias
- **Archive** — Arquivamento, mín. 180 dias (horas para reidratar)

Tipo	Cópias	Descrição
LRS	3	3 cópias no mesmo data center
ZRS	3	3 cópias em 3 zonas de disponibilidade
GRS	6	LRS local + LRS em região secundária
GZRS	6	ZRS primário + LRS em região secundária

Teorema CAP

Em sistemas distribuídos, é impossível garantir simultaneamente **Consistência**, **Disponibilidade** e **Tolerância a Partições**. Cada serviço faz trade-offs.

Portal → Pesquisar "Storage accounts" → + **Create**

Basics:

- 1 Resource Group: rg-curso-cloud
- 2 Storage account name: stcurso<seunome>2026 (globalmente único, minúsculas, sem hífens)
- 3 Region: Brazil South
- 4 Performance: Standard
- 5 Redundancy: LRS (mais barato para estudo)

Após criação:

Acesse o Storage Account → **Containers** → + **Container**

Nome: dados | Access level: Private

Clique no container → **Upload** → Selecione um arquivo

Passo 1 — Criar Storage Account

```
az storage account create -name stcursocloud2026  
-resource-group rg-curso-cloud  
-location brazilsouth -sku Standard_LRS
```

Passo 2 — Criar Container

```
az storage container create -name dados  
-account-name stcursocloud2026
```

Passo 3 — Upload do Arquivo

```
az storage blob upload -account-name stcursocloud2026  
-container-name dados  
-name arquivo.csv -file ./arquivo.csv
```

Rotação por Estações — Portal Azure (25 min cada)

Estação 1 — Blob Storage: Crie um container, faça upload de 3 arquivos, altere o nível de acesso de um blob para "Blob (anonymous read)", acesse a URL pública.

Estação 2 — Camadas de Acesso: Altere a camada de blobs (Hot→Cool→Archive). Use a Calculadora Azure para comparar custos mensais para 100 GB em cada camada.

Estação 3 — Redundância: Para 3 cenários dados pelo professor, identifique o tipo de redundância ideal e justifique.

Estação 4 — File Share: Crie um File Share e explore a opção "Connect" (gera script de montagem para Windows/Linux/macOS).

Objetivo da Aula

Compreender os tipos de bancos de dados NoSQL, suas aplicações e explorar o Azure Cosmos DB pelo Portal.

Metodologia Ativa

Hands-on Lab (GUI) — Criação, inserção e consulta de dados no Azure Cosmos DB via Portal.

- Bancos relacionais (SQL) são excelentes para dados estruturados e transações ACID
- Mas enfrentam desafios com:
 - Esquemas rígidos em domínios que mudam frequentemente
 - Escalabilidade horizontal (sharding é complexo)
 - Dados semi ou não-estruturados (JSON, documentos, grafos)
 - Volumes massivos com alta taxa de escrita (IoT, redes sociais)
- NoSQL oferece: esquema flexível, escala horizontal nativa, alta performance

Atenção

NoSQL **não substitui** SQL. São ferramentas complementares. A escolha depende do caso de uso.

Document Store

Documentos JSON flexíveis

Ex: MongoDB, Cosmos DB (SQL API)

Ideal para: catálogos, perfis de usuário

Column-Family (Wide Column)

Otimizado para leitura em colunas

Ex: Cassandra, HBase

Ideal para: séries temporais, IoT

Key-Value

Par chave-valor ultrarrápido

Ex: Redis, DynamoDB

Ideal para: cache, sessões, leaderboards

Graph

Nós + arestas (relacionamentos)

Ex: Neo4j, Cosmos DB (Gremlin)

Ideal para: redes sociais, fraude

- Banco de dados multimodelo, distribuído globalmente
- APIs suportadas: NoSQL (SQL-like), MongoDB, Cassandra, Gremlin, Table
- SLA de 99,999% de disponibilidade
- Latência inferior a 10ms no percentil 99
- Replicação automática multirregional (turnkey global distribution)
- 5 modelos de consistência configuráveis: Strong → Bounded Staleness → Session → Consistent Prefix → Eventual

Portal → Pesquisar "Azure Cosmos DB" → + **Create**

Selecione **Azure Cosmos DB for NoSQL**

Capacity mode: **Serverless** (custo mínimo para estudo)

Critério	SQL (Relacional)	NoSQL
Esquema	Fixo, definido previamente	Flexível, schema-on-read
Escalabilidade	Vertical (primária)	Horizontal (nativa)
Transações	ACID completo	BASE (eventual consistency)
Linguagem	SQL padronizado	Varia por tipo/produto
Relacionamentos	JOINS nativos	Desnormalização/embedding
Casos de uso	Financeiro, ERP, contábil	IoT, mobile, real-time, catálogos
Serviço Azure	Azure SQL Database	Cosmos DB, Redis Cache

Lab (Portal Azure): Azure Cosmos DB

- 1 Crie uma conta Cosmos DB (API NoSQL, Serverless, Brazil South).
- 2 Crie um database: loja e container: produtos (partition key: /categoria).
- 3 No **Data Explorer**, insira 10+ documentos JSON:

```
{"id": "1", "nome": "Notebook", "preco": 3500, "categoria": "eletronicos"}
```
- 4 Execute queries no Data Explorer:
 - `SELECT * FROM c`
 - `SELECT * FROM c WHERE c.preco > 100`
 - `SELECT c.categoria, COUNT(1) FROM c GROUP BY c.categoria`
- 5 Explore: **Metrics, Throughput, Replicate data globally**.

Reflexão

Compare a experiência com um banco relacional. Quando cada um é melhor?

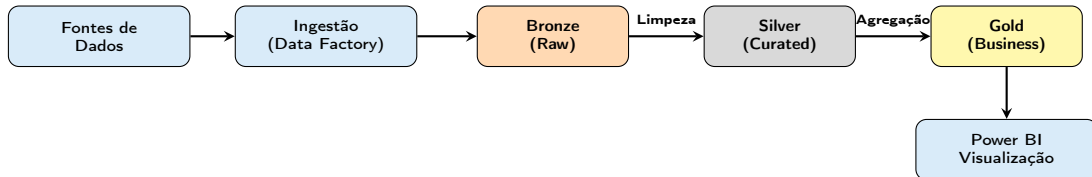
Objetivo da Aula

Explorar o ecossistema de dados no Azure: Data Lake, Data Warehouse, pipelines de dados e analytics.

Metodologia Ativa

Aprendizagem Baseada em Projetos — Desenho de uma arquitetura de dados para um caso real.

- **Azure Data Lake Storage Gen2** — Armazenamento massivo de dados brutos (combina Blob Storage + namespace hierárquico)
- **Azure Synapse Analytics** — Data warehouse moderno + analytics integrado (SQL + Spark em um só lugar)
- **Azure Data Factory** — ETL/ELT e orquestração de pipelines de dados (visual, low-code)
- **Azure Databricks** — Plataforma Apache Spark gerenciada (engenharia de dados + ML)
- **Azure Stream Analytics** — Processamento de dados em tempo real (queries SQL sobre streams)
- **Microsoft Purview** — Governança e catalogação de dados



Medallion Architecture

Bronze: Dados brutos, como chegaram (CSV, JSON, logs). **Silver:** Dados limpos, tipados, deduplicados. **Gold:** Dados agregados e prontos para consumo de negócio.

Projeto: Arquitetura de Dados para Rede de Farmácias

Em grupos de 4, projetem uma arquitetura de dados para uma rede de 500 farmácias que precisa:

- 1 Consolidar dados de vendas diárias (CSV de cada loja).
- 2 Armazenar histórico de 5 anos para análises.
- 3 Gerar dashboards de vendas em tempo real.
- 4 Integrar com sistema de recomendação de estoque.

Deliverables:

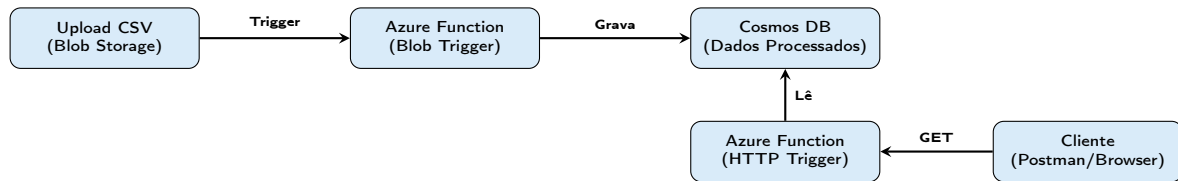
- Diagrama de arquitetura (draw.io) usando Medallion Architecture
- Lista de serviços Azure com justificativa
- Estimativa de custo (Calculadora Azure)

Objetivo da Aula

Integrar armazenamento e banco de dados em um cenário prático completo: pipeline simplificado Blob → Function → Cosmos DB.

Metodologia Ativa

Projeto Prático (GUI) — Construção de pipeline de dados simplificado usando exclusivamente o Portal Azure.



Etapas 1: Criar Storage Account + container "raw-data"

Etapas 2: Criar conta Cosmos DB (NoSQL, Serverless) + database + container

Etapas 3: Criar Function App (Portal → "Function App" → + Create)

- Runtime: Node.js ou Python
- Hosting: Consumption (Serverless)
- Vincular ao Storage Account criado

Etapas 4: Na Function App → Functions → + Create

- Trigger: Azure Blob Storage
- Path: raw-data/{name}
- Editar código diretamente no Portal (Code + Test)

Etapas 5: Criar segunda Function (HTTP trigger) para retornar dados como JSON

Projeto Prático: Pipeline de Dados

Em duplas, pelo Portal Azure:

- 1 Crie o Storage Account e o container "raw-data".
- 2 Crie a conta Cosmos DB com database e container.
- 3 Crie a Function App e implemente a função com Blob Trigger (código simplificado que lê o nome do arquivo e grava um documento no Cosmos DB).
- 4 Crie a função HTTP Trigger que retorna dados do Cosmos DB.
- 5 Teste: faça upload de um arquivo no Blob → verifique o documento no Cosmos DB Data Explorer → acesse a API HTTP.

Entrega

Prints de cada etapa + código das functions + print do Cosmos DB com os dados gravados.

Objetivo da Aula

Comparar na prática Azure SQL Database e Cosmos DB: provisionar ambos, inserir dados e executar queries para vivenciar as diferenças.

Metodologia Ativa

Experimentação Comparativa — Mesmo dataset em dois bancos diferentes, análise de trade-offs.

Conteúdo de Reserva

i Aula extra para aprofundamento prático em bancos de dados, caso haja tempo.

Portal → Pesquisar "SQL databases" → + **Create**

- 1 Resource Group: `rg-curso-cloud`
- 2 Database name: `db-curso-sql`
- 3 Server: Criar novo → nome único, admin login, senha
- 4 Compute + storage: **Basic** (5 DTU — mais barato)
- 5 Backup redundancy: **Locally-redundant**

Após criação: **Query editor (preview)** no menu lateral
Permite executar SQL diretamente no Portal!

Experimento: Mesmo Dado, Dois Bancos

Dataset: Lista de 20 produtos com id, nome, preço, categoria, estoque.

Azure SQL Database:

- 1 Criar tabela com schema definido (CREATE TABLE)
- 2 Inserir dados (INSERT INTO)
- 3 Queries: JOIN, GROUP BY, ORDER BY, WHERE

Azure Cosmos DB (já criado):

- 1 Inserir documentos JSON (sem schema prévio)
- 2 Queries com sintaxe SQL-like
- 3 Tentar equivalentes de JOIN e GROUP BY

Comparar: facilidade, flexibilidade, tipos de queries possíveis.

Hands-on Comparativo

Individualmente:

- 1 Provisione um Azure SQL Database (tier Basic).
- 2 No Query Editor, crie uma tabela produtos e insira 10+ registros.
- 3 Execute queries com WHERE, GROUP BY, ORDER BY.
- 4 No Cosmos DB (já provisionado), insira os mesmos dados como JSON.
- 5 Execute queries equivalentes.
- 6 Preencha a tabela comparativa fornecida pelo professor: facilidade de modelagem, queries suportadas, performance percebida, custo estimado.

Entrega

Tabela comparativa preenchida + reflexão: "Qual banco eu usaria para meu projeto e por quê?"

Objetivo da Aula

Comparar as principais plataformas cloud (Azure, AWS, GCP), seus diferenciais, serviços equivalentes e critérios de escolha.

Metodologia Ativa

World Café — Mesas temáticas por provedor cloud, rotação e construção coletiva.

Categoria	Azure	AWS	GCP
VMs	Virtual Machines	EC2	Compute Engine
Serverless	Azure Functions	Lambda	Cloud Functions
Kubernetes	AKS	EKS	GKE
NoSQL	Cosmos DB	DynamoDB	Firestore / Bigtable
Object Store	Blob Storage	S3	Cloud Storage
API Gateway	API Management	API Gateway	Apigee
Relacional	Azure SQL	RDS / Aurora	Cloud SQL / AlloyDB
IA / ML	Azure AI / OpenAI	SageMaker	Vertex AI
IaC	Bicep / ARM	CloudFormation	Deployment Manager
Diferencial	Ecossistema Microsoft	Maior market share	Big Data / ML

- **Ecosistema existente** — Já usa Microsoft (Office, AD)? Azure tem vantagem
- **Serviços específicos** — ML/Big Data? GCP. Mais opções? AWS
- **Regiões e compliance** — Disponibilidade em Brazil South/Southeast
- **Custo** — Modelos de pricing variam (reserved, spot, committed use)
- **Comunidade e talento** — Facilidade de encontrar profissionais
- **Suporte e SLA** — Garantias contratuais e suporte local
- **Vendor lock-in** — Grau de dependência de serviços proprietários

Realidade do Mercado

Muitas empresas adotam multcloud ou híbrido. A habilidade de **entender conceitos** (não só ferramentas) permite transitar entre provedores.

World Café: Plataformas Cloud (3 mesas, 3 rodadas de 20 min)

Mesa 1 — Azure: Liste 10 serviços principais e mapeie equivalentes em AWS e GCP. Use a documentação oficial.

Mesa 2 — AWS: Compare preços de 3 serviços (VM, Storage, Serverless) entre Azure e AWS usando as calculadoras oficiais.

Mesa 3 — GCP: Identifique 3 diferenciais do GCP para Big Data e ML. Liste serviços que não têm equivalente direto nos outros.

A cada rodada, o grupo rotaciona e **complementa** o trabalho anterior.

Entrega

Quadro comparativo consolidado (contribuição de todos os grupos).

Objetivo da Aula

Configurar um ambiente Azure organizado com boas práticas: hierarquia de recursos, nomenclatura, tags, RBAC, budgets e políticas.

Metodologia Ativa

Laboratório com Checklist — Setup guiado passo a passo no Portal Azure.

- **Microsoft Entra ID (antigo Azure AD)** — Identidades e diretório
- **Management Groups** — Agrupamento de assinaturas (governança em escala)
- **Subscriptions (Assinaturas)** — Contêiner de billing e limites de recursos
- **Resource Groups** — Agrupamento lógico (ciclo de vida compartilhado)
- **Resources** — Serviços individuais (VMs, DBs, Storage, etc.)

Boas Práticas

- **Nomenclatura padronizada:** <tipo>-<projeto>-<ambiente>-<região>
Ex: rg-ecommerce-prod-brazilsouth, vm-api-dev-eastus
- **Tags obrigatórias:** Environment, Project, Owner, CostCenter
- **RBAC:** Princípio do menor privilégio (least privilege)
- **Locks:** CanNotDelete em recursos críticos de produção

Portal → Pesquisar "Cost Management + Billing"

- **Cost analysis:** Visualize gastos por serviço, região, tag, período
- **Budgets:** Crie alertas de orçamento
→ + Add → Budget amount: R\$ 50 → Alert at 80% e 100%
- **Advisor recommendations:** Sugestões de economia
Portal → Advisor → Cost → Revise recomendações

Dica para Alunos

Configure um budget de R\$ 20–50 com alerta por e-mail a 50% para evitar surpresas na fatura!

Lab: Setup Completo do Ambiente (Checklist)

Individualmente, no Portal Azure:

- 1 Verifique sua assinatura ativa (Overview da Subscription).
- 2 Crie 3 Resource Groups: rg-prod, rg-dev, rg-test.
- 3 Aplique tags em cada RG: Environment, Aluno, Disciplina.
- 4 Configure um **Budget** de R\$ 50/mês com alerta a 80%.
- 5 Acesse o **Azure Advisor** e anote 3+ recomendações.
- 6 Explore **Cost Analysis**: filtre gastos por Resource Group.
- 7 Aplique um **Lock** (CanNotDelete) no rg-prod.
- 8 Tente excluir o rg-prod e observe o comportamento.

Entrega

Print de cada etapa + reflexão sobre governança em nuvem.

Objetivo da Aula

Realizar o deploy de uma aplicação web no Azure App Service via Portal, com integração ao GitHub para deploy contínuo.

Metodologia Ativa

Pair Programming — Deploy colaborativo em duplas.

- Serviço PaaS para hospedar aplicações web, APIs e backends mobile
- Runtimes: .NET, Node.js, Python, Java, PHP, Ruby, Go
- Recursos incluídos: SSL/TLS automático, custom domains, autoscaling
- Opções de deploy: GitHub, Azure DevOps, Bitbucket, Local Git, ZIP, Docker

Estratégias de Deploy

- **Deployment Slots** — Ambientes separados (staging, production) com swap instantâneo (zero downtime)
- **Continuous Deployment** — Deploy automático a cada push no GitHub
- **Manual Deploy** — Upload de código/ZIP pelo Portal

1. Criar App Service:

Portal → App Services → + Create → Web App
Name: único | Runtime: Node 18 LTS | Plan: Free F1

2. Configurar Deploy do GitHub:

App Service → **Deployment Center** (menu lateral)
Source: **GitHub** → Authorize → Selecionar repositório e branch
O Azure cria automaticamente um workflow GitHub Actions!

3. Testar:

Faça um commit no repositório → observe o deploy automático
App Service → Overview → clique na **Default domain** (URL)

4. Monitorar:

App Service → **Log stream** (logs em tempo real)
App Service → **Metrics** (CPU, memória, requests)

Passo 1 — Criar Web App

```
az webapp create -name app-meusite  
-resource-group rg-curso-cloud  
-plan plan-free -runtime "NODE:18-lts"
```

Passo 2 — Conectar ao Repositório

```
az webapp deployment source config  
-name app-meusite -resource-group rg-curso-cloud  
-repo-url https://github.com/user/repo  
-branch main
```

Passo 3 — Criar Slot (Staging)

```
az webapp deployment slot create  
-name app-meusite -resource-group rg-curso-cloud  
-slot staging
```

Passo 4 — Swap para Produção

```
az webapp deployment slot swap  
-name app-meusite -resource-group rg-curso-cloud  
-slot staging
```

Pair Programming: Deploy Completo

Em duplas:

- 1 Crie um repositório no GitHub com uma aplicação web simples (sugestão: HTML estático ou Node.js Express com "Hello Cloud!").
- 2 No Portal, crie um App Service (tier Free F1).
- 3 No **Deployment Center**, conecte ao repositório GitHub.
- 4 Aguarde o deploy e acesse a URL pública.
- 5 Faça uma alteração no código, commit, e observe o **redesploy automático**.
- 6 Explore: Metrics, Log stream, Configuration (variáveis de ambiente).

Entrega

URL da aplicação funcionando + link do repositório GitHub + print do Deployment Center mostrando deploy bem-sucedido.

Objetivo da Aula

Iniciar o desenvolvimento do projeto final integrando todos os conceitos do curso em uma solução cloud completa no Azure.

Metodologia Ativa

Aprendizagem Baseada em Projetos (PBL) — Desenvolvimento guiado com checkpoints e mentoria do professor.

Requisitos Obrigatórios

Em grupos de 3–4 alunos, desenvolvam uma solução cloud com:

- 1 **Arquitetura:** Diagrama completo da solução no Azure
- 2 **Rede:** VNet com pelo menos 2 sub-redes + NSG
- 3 **Aplicação:** API REST ou Web App deployada no App Service ou Functions
- 4 **Dados:** Pelo menos 2 serviços de dados (Blob Storage + Cosmos DB ou SQL Database)
- 5 **Integração:** API Management ou Logic Apps
- 6 **CI/CD:** Deploy via GitHub (Deployment Center)
- 7 **Governança:** Resource Groups com tags, budget configurado
- 8 **Custos:** Estimativa mensal com Calculadora Azure

Temas para o Projeto

- ❶ **E-commerce Serverless** — Catálogo com Azure Functions + Cosmos DB + Blob Storage para imagens
- ❷ **Sistema de Monitoramento IoT** — Ingestão de dados de sensores + armazenamento + dashboard
- ❸ **Plataforma de Gestão de Tarefas** — API REST + banco relacional + autenticação + CI/CD
- ❹ **Data Pipeline para Analytics** — Ingestão CSV via Blob + processamento via Functions + visualização
- ❺ **Portal de Notícias/Blog** — App Service + Cosmos DB + Storage para mídia + CDN
- ❻ **Sistema de Agendamento** — Logic Apps + Cosmos DB + notificações por e-mail

Aula 23 — Checkpoints

Checkpoint 1 (30 min): Definição do tema e formação dos grupos. Apresentar ao professor: tema, membros, escopo.

Checkpoint 2 (45 min): Diagrama de arquitetura (rascunho no draw.io). Validar com o professor: serviços escolhidos, viabilidade.

Checkpoint 3 (45 min): Início do provisionamento no Portal Azure. Criar Resource Group, primeiros recursos e começar o código.

Importante

Use **apenas serviços gratuitos ou de baixo custo** (Free tier, Serverless, Basic). Configurem budget alert!

Objetivo da Aula

Apresentar e defender o projeto final perante a turma e o professor, demonstrando competência em arquitetura e serviços cloud.

Formato

15 minutos por grupo (apresentação + demo ao vivo) + **5 minutos** de perguntas da turma e do professor.

Estrutura Sugerida (15 min)

- 1 **Contexto e Problema (2 min):** Qual problema a solução resolve?
- 2 **Arquitetura (3 min):** Diagrama, serviços utilizados, justificativas
- 3 **Demo ao Vivo (5 min):** Mostrar a solução funcionando no Portal Azure — recursos provisionados, aplicação rodando, dados no banco
- 4 **Rede e Segurança (2 min):** VNet, NSGs, governança aplicada
- 5 **Custos e Escalabilidade (2 min):** Estimativa mensal, como escalaria para produção
- 6 **Lições Aprendidas (1 min):** Desafios enfrentados e como resolveram

Entregáveis do Projeto Final

- ❶ **Diagrama de Arquitetura** — draw.io, Visio ou equivalente
- ❷ **Documento Técnico** (2–3 páginas):
 - Descrição da solução e problema resolvido
 - Lista de serviços Azure utilizados com justificativa
 - Configuração de rede (VNets, sub-redes, NSGs)
 - Estratégia de dados (armazenamento + banco)
 - Estimativa de custos (print da Calculadora Azure)
 - Lições aprendidas
- ❸ **Repositório GitHub** — Código-fonte com README
- ❹ **Prints do Portal Azure** — Recursos provisionados

Objetivo

Introduzir o conceito de IaC e o Azure Bicep como ferramenta declarativa para provisionamento automatizado de infraestrutura.

Metodologia Ativa

Coding Dojo — Escrita colaborativa de templates Bicep.

Conteúdo de Reserva

i Aula de reserva para caso o professor adiante o conteúdo principal.

O que é Infrastructure as Code?

- Definir infraestrutura em arquivos de código versionáveis
- Benefícios:
 - **Reprodutibilidade** — Mesmo template, mesmo resultado
 - **Versionamento** — Git para infraestrutura
 - **Automação** — CI/CD para infraestrutura
 - **Documentação** — O código é a documentação
 - **Auditabilidade** — Quem mudou o quê, quando
- Ferramentas: **Bicep** (Azure nativo), Terraform (multi-cloud), ARM Templates, Pulumi, CloudFormation (AWS)

```
1 // Parametros
2 param location string = 'brazilsouth'
3 param appName string = 'app-bicep-demo'
4
5 // App Service Plan
6 resource appPlan 'Microsoft.Web/serverfarms@2022-09-01' = {
7   name: 'plan-${appName}'
8   location: location
9   sku: {
10     name: 'F1' // Free tier
11     tier: 'Free'
12   }
13   kind: 'linux'
14   properties: {
15     reserved: true
16   }
17 }
18
19 // Web App
20 resource webApp 'Microsoft.Web/sites@2022-09-01' = {
21   name: appName
22   location: location
23   properties: {
24     serverFarmId: appPlan.id
25   }
26 }
```

main.bicep — Exemplo de template

Coding Dojo: Escrevendo Bicep

Em formato de *dojo* (um piloto, um copiloto, plateia rotaciona):

- ❶ Escreva um template Bicep que provisione:
 - 1 Resource Group
 - 1 VNet com 2 sub-redes
 - 1 Storage Account
 - 1 App Service (Free F1)
- ❷ Use a extensão Bicep do VS Code para validação.
- ❸ Opcional: deploy via **Portal** → Deploy a custom template → Build your own template → cole o Bicep.

Portal → "Deploy a custom template" → Build your own → Editor aceita Bicep/ARM!

Objetivo

Compreender os pilares de observabilidade (logs, métricas, traces) e configurar monitoramento no Azure.

Metodologia Ativa

Investigação Guiada — Diagnosticar problemas em uma aplicação usando ferramentas de monitoramento do Azure.

Conteúdo de Reserva

i Aula extra sobre monitoramento, caso haja tempo disponível.

Logs

- Registros de eventos
- Texto estruturado/JSON
- Debugging e auditoria
- Azure Monitor Logs
- Log Analytics Workspace

Métricas

- Dados numéricos ao longo do tempo
- CPU, memória, latência, throughput
- Alertas baseados em thresholds
- Azure Monitor Metrics
- Dashboards customizáveis

Traces

- Rastreamento de requisições entre serviços
- Distributed tracing
- Identificar gargalos
- Application Insights
- End-to-end correlation

- **Azure Monitor** — Plataforma unificada de monitoramento
- **Application Insights** — APM (Application Performance Monitoring)
 - Detecção automática de anomalias
 - Mapa de dependências entre serviços
 - Métricas de performance (tempo de resposta, taxa de falha)
- **Log Analytics** — Consultas KQL sobre logs centralizados
- **Azure Alerts** — Regras de alerta por métrica, log ou atividade
- **Azure Dashboards** — Painéis customizáveis no Portal
- **Azure Service Health** — Status dos serviços Azure

App Service → **Application Insights** → Enable → Cria workspace automaticamente
Após habilitar, acesse Application Insights → Live Metrics, Failures, Performance

Investigação: Diagnosticando Problemas

Usando o App Service criado anteriormente:

- ❶ Habilite **Application Insights** no App Service (via Portal).
- ❷ Gere tráfego acessando a URL da aplicação várias vezes.
- ❸ No Application Insights, explore:
 - **Live Metrics** — Métricas em tempo real
 - **Performance** — Tempo de resposta das requisições
 - **Failures** — Erros (force um erro 404 acessando URL inexistente)
- ❹ Crie um **Alert Rule**: alerta quando tempo de resposta > 2 segundos.
- ❺ Crie um **Dashboard** com 3 métricas no Portal.

Objetivo

Compreender os serviços de identidade e segurança do Azure: Entra ID, RBAC, Key Vault e boas práticas.

Metodologia Ativa

Estudo de Caso + Lab — Análise de incidentes de segurança e configuração de segurança no Portal.

Conteúdo de Reserva

i Aula extra sobre segurança cloud, para caso haja tempo disponível.

- Serviço de identidade e acesso da Microsoft na nuvem
- Autenticação: Single Sign-On (SSO), MFA (Multi-Factor Authentication)
- Autorização: RBAC (Role-Based Access Control)
- Integração com milhares de aplicações SaaS
- Conditional Access: políticas baseadas em contexto (local, dispositivo, risco)

RBAC no Azure

- **Roles built-in:** Owner, Contributor, Reader (+ centenas de roles específicas)
- **Scope:** Management Group → Subscription → Resource Group → Resource
- **Princípio:** Least Privilege — conceda apenas o necessário

- **Azure Key Vault** — Armazenamento seguro de:
 - Secrets (senhas, connection strings, API keys)
 - Keys (chaves de criptografia)
 - Certificates (certificados SSL/TLS)
- **Microsoft Defender for Cloud** — Postura de segurança unificada
 - Secure Score: nota de segurança do ambiente
 - Recomendações acionáveis
 - Compliance com frameworks (CIS, NIST, LGPD)

Portal → Pesquisar "Key vaults" → + Create

Após criação → Secrets → + Generate/Import → Armazene uma connection string

App Service → Configuration → Key Vault reference para secrets

Lab: Segurança no Azure

- 1 Acesse **Microsoft Entra ID** no Portal → Users → observe a estrutura de identidades.
- 2 No Resource Group, vá em **Access Control (IAM)** → View role assignments → observe as roles atribuídas.
- 3 Crie um **Key Vault** → Armazene 2 secrets (ex: senha de banco, API key fictícia).
- 4 Acesse **Microsoft Defender for Cloud** → observe o Secure Score e 3 recomendações.
- 5 Documente: quais vulnerabilidades foram identificadas? Quais ações são recomendadas?

Objetivo

Revisar os conceitos fundamentais de nuvem alinhados à certificação **Microsoft Azure Fundamentals (AZ-900)**.

Metodologia Ativa

Quiz Gamificado + Revisão — Simulado interativo com questões no estilo da prova.

Conteúdo de Reserva

i Aula de revisão e preparação para certificação, ideal para encerramento do curso.

- **Público-alvo:** Iniciantes em nuvem e Azure
- **Formato:** 40–60 questões de múltipla escolha
- **Duração:** 85 minutos
- **Nota mínima:** 700/1000
- **Custo:** Gratuito para estudantes (Microsoft Imagine Academy)
- **Domínios cobrados:**
 - 1 Conceitos de nuvem (25–30%)
 - 2 Arquitetura e serviços Azure (35–40%)
 - 3 Gerenciamento e governança (30–35%)

Recurso Gratuito

Microsoft Learn: <https://learn.microsoft.com/training/paths/az-900-describe-cloud-concepts/>

Conceitos de Nuvem

- IaaS / PaaS / SaaS
- Nuvem pública / privada / híbrida
- CapEx vs. OpEx
- Elasticidade e escalabilidade
- Alta disponibilidade e tolerância a falhas
- Modelo de responsabilidade compartilhada

Serviços e Governança

- VMs, App Service, Functions
- VNets, NSGs, Load Balancer
- Storage, Cosmos DB, SQL DB
- Entra ID, RBAC, Key Vault
- Azure Monitor, Advisor
- Cost Management, Tags, Policies

Quiz Gamificado: Simulado AZ-900

Formato: O professor projeta questões no estilo AZ-900. Alunos respondem individualmente (Kahoot, Mentimeter ou papel).

Rodada 1 (20 min): 15 questões — Conceitos de Nuvem

Rodada 2 (20 min): 15 questões — Serviços Azure

Rodada 3 (20 min): 15 questões — Governança e Segurança

Após cada rodada: revisão das respostas com explicação detalhada.

Dica

O Microsoft Learn oferece **módulos gratuitos de preparação para o AZ-900** com labs práticos incluídos (sandbox Azure gratuito).

Documentação e Estudo

- Microsoft Learn: <https://learn.microsoft.com/azure/>
- Azure Architecture Center: <https://learn.microsoft.com/azure/architecture/>
- Azure for Students: <https://azure.microsoft.com/free/students/>
- Azure Charts (comparativo visual): <https://azurecharts.com/>

Certificações Recomendadas

- **AZ-900** — Azure Fundamentals (nível iniciante)
- **AZ-204** — Developing Solutions for Azure (nível associado)
- **AZ-305** — Designing Azure Infrastructure (nível expert)

Obrigado!

Dúvidas e Contato

ronierison.maciел@pe.senac.br

"A nuvem não é um destino, é uma jornada de transformação."